

DSHUB BUILD2EARN – COHORT C

COLLEGE FOOTBALL FANTASY

CYBERSECURITY REQUIREMENTS DOCUMENT (CSRD)

Project	DSHub Build2Earn – Cohort C
Team	Team 4
Product Manager	Aliyu Muhammed
Role	Cybersecurity Engineer
Version	1.0
Date	September 2026
Status	Draft – For Team Review

Prepared by: Cybersecurity Engineering – Team 4

Table of Contents

1. Purpose
2. Security Objectives
3. Authentication Requirements
4. Authorization & Role-Based Access Control
5. API Security Requirements
6. Fantasy Team & Salary-Cap Security
7. Scoring & Ranking Integrity
8. Gameweek & Lineup Security
9. Player Data Integrity
10. Database Security
11. Data Privacy
12. Input Validation & Secure Coding
13. Administrative Security
14. Audit Logging
15. Security Monitoring
16. Threat Modelling
17. Security Testing
18. Secure Development & Dependencies
19. Deployment Security
20. Incident Response
21. Security Requirements Traceability
22. Security Acceptance Criteria
23. Priority Levels
24. Open Cybersecurity Decisions
25. Approval & Sign-Off

Document Control

Version Control

Version	Date	Author/Owner	Change Description	Status
1.0	September 2026	Cybersecurity Engineer – Team	Initial cybersecurity requirements draft	Draft – For Team Review

1. Purpose

This Cybersecurity Requirements Document defines the security requirements for the College Football Fantasy platform.

The objective is to protect user accounts, fantasy teams, player data, scoring, rankings, APIs, administrative functions, and personal information against unauthorized access, manipulation, abuse, and data loss.

Security requirements should be considered throughout the product lifecycle, from development and testing through deployment and ongoing monitoring.

2. Security Objectives

Confidentiality – Protect user and system information from unauthorized access.

Integrity – Ensure player data, fantasy points, rankings, salary values, teams, and other important information cannot be improperly modified.

Availability – Keep the platform available and reliable, particularly during active gameweeks and high-traffic periods.

Accountability – Maintain appropriate records of important administrative and security activities.

3. Authentication Requirements

The system shall require authentication before users can access protected account functions.

The system shall support secure registration and email verification where applicable.

Passwords shall be stored using a strong, modern password-hashing algorithm and never in plaintext.

The system shall provide secure password reset and protection against repeated failed login attempts.

Authentication tokens and sessions shall be securely managed and protected from unauthorized disclosure.

Multi-factor authentication should be considered for administrators and other privileged accounts.

4. Authorization & Role-Based Access Control

The application shall implement Role-Based Access Control (RBAC). Users must only perform actions permitted for their role.

Proposed roles: Standard User – manage own account, fantasy team and leagues; League Administrator – manage permitted league functions; System Administrator – manage players, gameweeks, users and system functions; Privileged Administrator – perform sensitive administrative/security operations.

Authorization shall be enforced on the backend/API. Users must not be able to access or modify another user's private resources. The principle of least privilege shall be followed.

5. API Security Requirements

Protected REST APIs shall require appropriate authentication and authorization.

Incoming API data shall be validated. Malformed, unauthorized, or abusive requests shall be rejected.

Appropriate rate limiting shall be implemented. API credentials and secrets shall be protected.

API responses shall not unnecessarily expose sensitive information. Secure error handling shall be used.

HTTPS/TLS shall protect API communications.

Security controls must be enforced server-side rather than relying solely on frontend restrictions.

6. Fantasy Team & Salary-Cap Security

The backend shall independently validate fantasy team creation and modification.

The system shall verify player eligibility, squad composition, and that the total virtual salary does not exceed \$100M.

Users shall not be able to manipulate player prices or modify another user's squad through client-side requests.

Transfers shall be validated before being committed.

Client-supplied values shall be treated as untrusted.

7. Scoring & Ranking Integrity

Scoring and ranking integrity is a high-priority security concern because these values directly affect competition.

Performance data shall come from an approved/trusted source and be validated before scoring.

Scoring shall be performed by authorized backend processes. Ordinary users shall not be able to directly modify fantasy points or rankings.

Scoring configuration changes and manual score corrections shall be restricted to authorized administrators and logged.

8. Gameweek & Lineup Security

The backend shall enforce gameweek deadlines and lineup locks.

Before accepting lineup changes, the server should verify the current gameweek, applicable deadline, player fixture status, user authorization, and squad validity.

Once a lineup is locked, unauthorized changes shall be rejected. Frontend controls alone shall not enforce the deadline.

9. Player Data Integrity

Player information and performance data shall be validated before entering the scoring process.

Modification of player data and pricing shall be restricted to authorized personnel or processes.

The system should monitor unexpected changes to critical player data and maintain appropriate administrative records.

10. Database Security

The PostgreSQL database shall use strong authentication, least-privilege accounts, encrypted connections, and encryption at rest where supported.

Database credentials shall be securely stored. Parameterized queries/prepared statements shall be used to reduce SQL injection risk.

Administrative database access shall be restricted. Backup and recovery procedures shall be defined and tested.

11. Data Privacy

The application shall collect and process only personal information necessary for its functions.

The team should document what personal information is collected, why it is collected, where it is stored, who can access it, retention requirements, and applicable deletion/management processes.

Personal information shall not be unnecessarily exposed through APIs, logs, dashboards, or error messages. Required privacy and legal reviews should be completed for target markets.

12. Input Validation & Secure Coding

All user-controlled input shall be treated as untrusted.

Appropriate input validation, data-type validation, length restrictions, server-side validation, and output encoding shall be implemented where applicable.

The development team should consider injection, broken access control, authentication weaknesses, security misconfiguration, insecure API behavior, and sensitive-data exposure.

13. Administrative Security

Administrative functions shall receive additional protection through strong authentication, appropriate authorization, least privilege, audit logging, and monitoring.

Sensitive operations include modifying player data or prices, managing gameweeks, correcting scores, managing users, and changing security settings.

14. Audit Logging

Appropriate audit records shall be maintained for important administrative and security events, including privileged logins, failed privileged logins, role changes, player-data changes, score corrections, gameweek changes, and security configuration changes.

Logs shall not unnecessarily contain passwords, authentication tokens, or other sensitive information. Audit-log access shall be restricted to authorized personnel.

15. Security Monitoring

The application should monitor repeated failed logins, unusual authentication activity, abnormal API patterns, unauthorized access attempts, unexpected administrative actions, suspicious changes to critical data, and relevant application/server security events.

Appropriate alerts should be considered for high-risk events.

16. Threat Modelling

A security threat model should be maintained for the application.

Initial Threat Register

Asset	Threat	Risk	Primary Control
User accounts	Account takeover	High	Secure authentication + rate limiting
Fantasy teams	Unauthorized modification	High	Backend authorization
Fantasy points	Score manipulation	Critical	Server-side scoring
Rankings	Ranking manipulation	High	Controlled scoring service
Player data	Data tampering	High	Validation + access control
Salary values	Price manipulation	High	Server-side validation
Admin panel	Unauthorized access	Critical	RBAC + strong authentication
API	Abuse/unauthorized access	High	Authentication + rate limiting
Personal data	Unauthorized disclosure	High	Access controls + encryption
Database	Unauthorized access	Critical	Least privilege + encryption
Audit logs	Unauthorized modification	High	Restricted access + integrity controls

17. Security Testing

Security testing shall be included before production deployment.

Testing should include authentication, authorization, API security, input validation, access control, session/token security, database security, vulnerability assessment, dependency/configuration review, and security regression testing.

Authorized penetration testing may be performed where approved and within the defined testing scope.

18. Secure Development & Dependencies

The engineering team should keep dependencies updated, review third-party libraries for known security issues, protect secrets, avoid committing credentials to source control, use separate environments, and review security-sensitive code before production.

19. Deployment Security

The deployment pipeline should follow Development → Testing → Staging → Production.

Before production release, QA approval, security review, security testing, data-validation review, and product approval should be completed.

Production credentials and secrets shall not be unnecessarily exposed to developers or stored directly in source code.

20. Incident Response

The team should establish a basic process covering identification, initial assessment, containment, investigation, remediation, recovery, documentation, and lessons learned.

Examples include account compromise, unauthorized data access, API abuse, data manipulation, privileged-account compromise, and significant vulnerability discovery.

21. Security Requirements Traceability

Security requirements shall map to relevant PRD requirements and technical components.

Product Requirement	Technical Component	Security Requirement
Account	Authentication Service	Authentication & session security
Build Team	Squad Management	Authorization + server-side validation
Player Search	Player API	API security + input validation
Earn Points	Scoring Service	Data integrity + scoring protection
Manage Lineup	Lineup Service	Authorization + deadline enforcement
Leagues	League Service	Access control
Rankings	Ranking Service	Integrity protection
Notifications	Notification Service	Secure data handling
Admin Management	Admin Panel	RBAC + audit logging
Player Data	Data Integration	Data validation + integrity
User Data	Database	Privacy + database security

22. Security Acceptance Criteria

The MVP should not be considered security-ready until secure registration/authentication, password hashing, protected APIs, server-side authorization, resource isolation, server-side salary-cap enforcement, scoring integrity, lineup-deadline enforcement, administrative access controls, audit logging, input validation, HTTPS/TLS, appropriate data protection, and security testing are in place.

Identified high-risk vulnerabilities should be addressed or formally accepted by the appropriate project authority before production.

23. Priority Levels

Critical: Authentication, authorization/RBAC, scoring integrity, admin security, API access control, sensitive-data protection.

High: Rate limiting, audit logging, input validation, database security, threat modelling, security monitoring.

Medium: Advanced security analytics, additional monitoring, and improvements identified during testing.

24. Open Cybersecurity Decisions

Final authentication architecture; JWT/session strategy; MFA requirements for administrators; RBAC structure; API rate limits; data-retention requirements; logging/monitoring solution; backup/recovery strategy; security-testing scope; authorized penetration-testing scope; data-provider security requirements; privacy/legal requirements for target markets.

25. Approval & Sign-Off

This document is submitted for review and alignment with the Product Manager, Engineering team, and other relevant project stakeholders. Approval indicates that the listed cybersecurity requirements are accepted as the current security baseline, subject to future technical decisions and documented revisions.

Approval & Sign-Off

Role	Name	Signature	Date
Product Manager	Aliyu Muhammed	_____	_____
Cybersecurity Engineer	_____	_____	_____
Engineering Lead	_____	_____	_____
QA / Security Reviewer	_____	_____	_____